

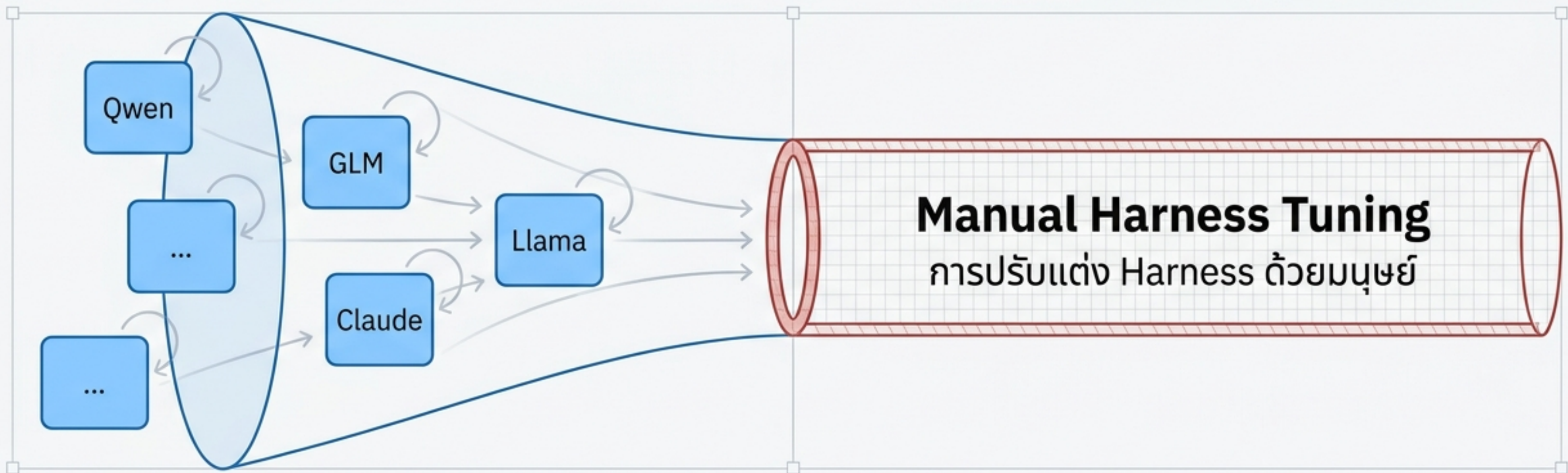


Self-Harness: เมื่อ AI สร้างและปรับปรุงพิมพ์เขียวของตัวเอง

กระบวนการใหม่ที่ให้ Agent เรียนรู้และแก้ไขระบบควบคุม (Harness) ของตนเองผ่าน Execution Traces

For a conscious being, to exist is to change, to change is to mature, to mature is to go on creating oneself endlessly.
— Henri Bergson, *Creative Evolution*

คอบวดของการพัฒนา AI Agent



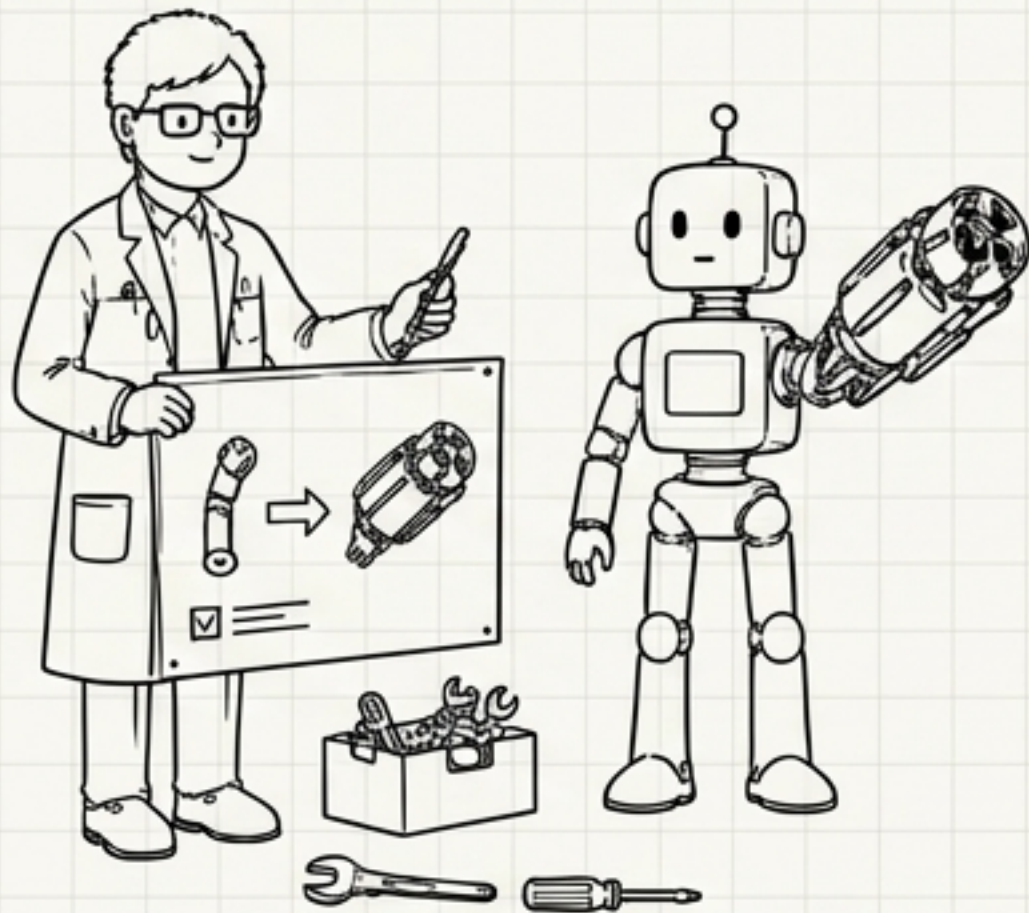
Harness คืออะไร?

โครงสร้างระบบที่ล้อมรอบและควบคุม LLM (Prompts, Tools, Memory, Policies) โมเดลเดียวกันอาจทำงานได้ดีหรือแย่ลงอย่างสิ้นเชิงขึ้นอยู่กับ Harness นี้

ปัญหา: โมเดลแต่ละตัวมีพฤติกรรม จุดอ่อน และการตอบสนองที่ต่างกัน การใช้ผู้เชี่ยวชาญ (Human Experts) นั่งปรับแต่ง Harness ให้เข้ากับโมเดลใหม่ทุกตัว เป็นกระบวนการที่สิ้นเปลืองและไม่สามารถขยายสเกลได้ (Scalable) ในยุคที่ LLM พัฒนาอย่างก้าวกระโดด

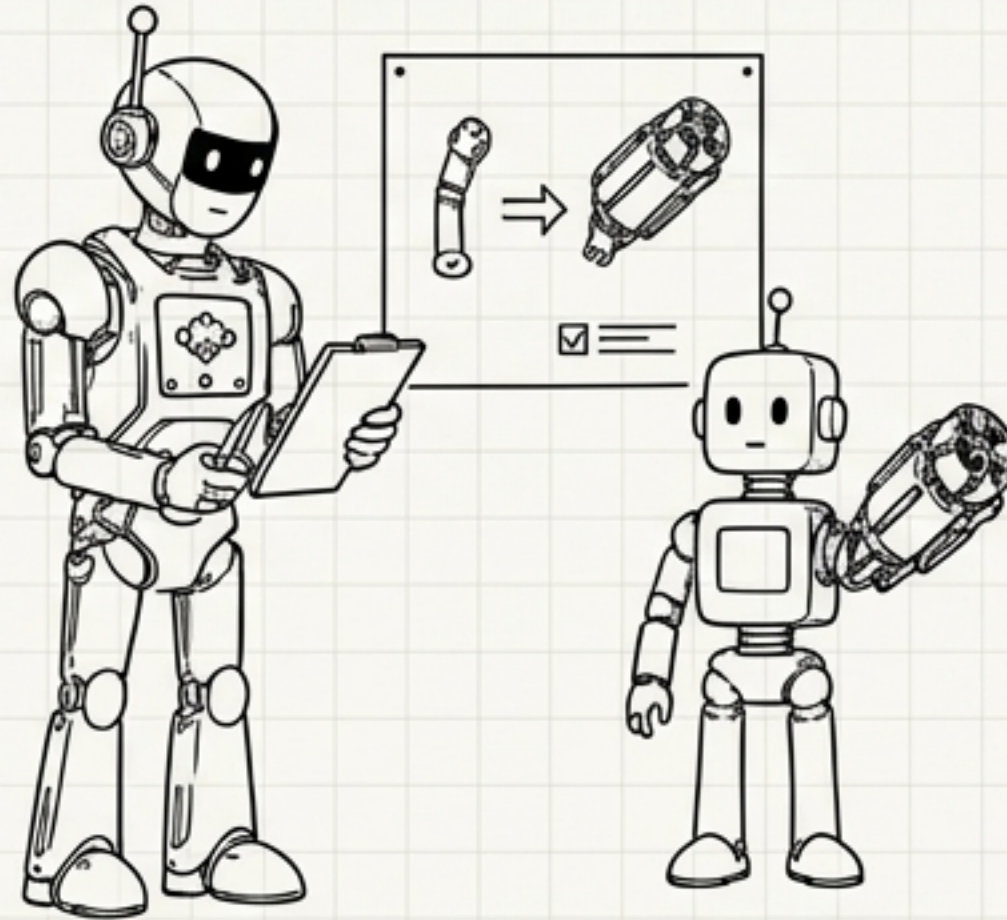
3 ยุคของการออกแบบโครงสร้าง Agent (Three Eras of Agent Design)

Human Harness Engineering (มนุษยวิศวกรรม)



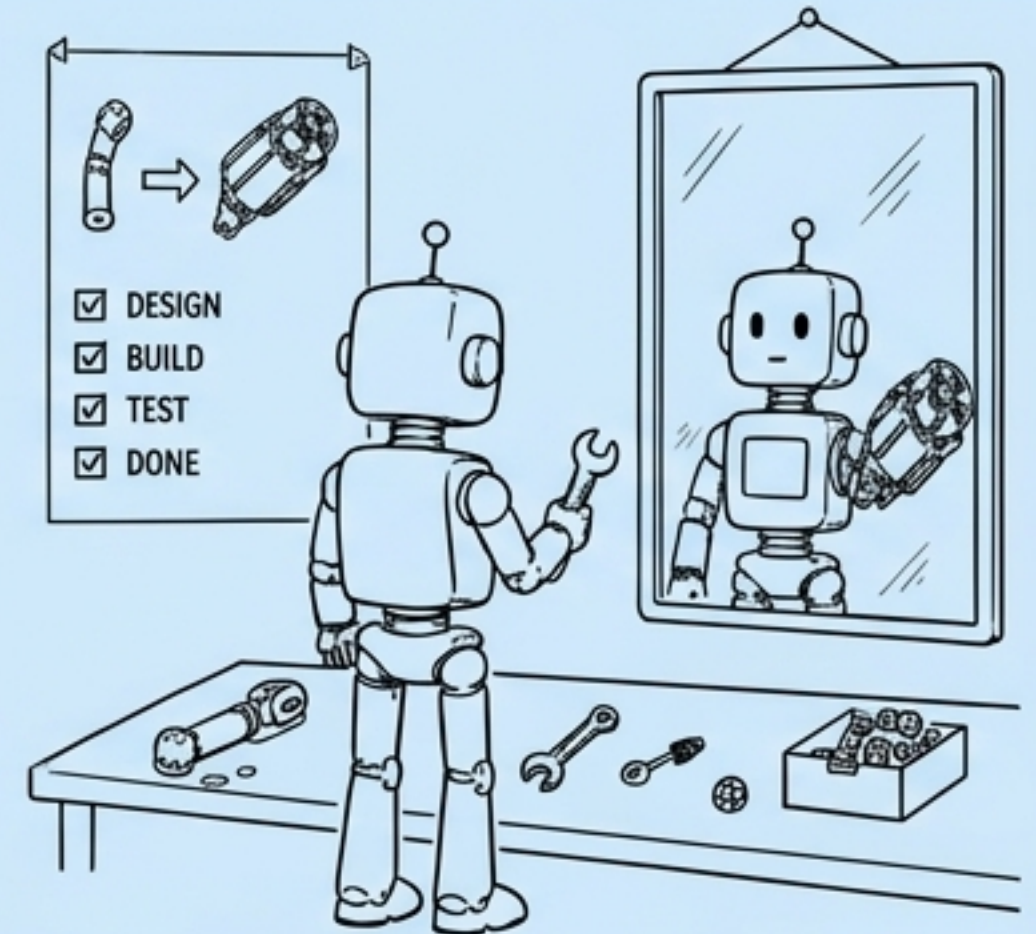
- **ผู้ขับเคลื่อน:** ผู้เชี่ยวชาญที่เป็นมนุษย์ (Human Experts)
- **ข้อจำกัด:** ไม่สามารถขยายสเกลได้ (Unscalable), มีต้นทุนสูงในการปรับแต่งตามโมเดล

Meta-Harness (ระบบพี่เลี้ยง)



- **ผู้ขับเคลื่อน:** โมเดลภายนอกที่ฉลาดกว่า (Stronger External Agent)
- **ข้อจำกัด:** พึ่งพาระบบภายนอก, มีค่าใช้จ่ายสูง, อาจไม่เข้าใจจุดอ่อนเฉพาะตัวของโมเดลเป้าหมาย

Self-Harness (กระจกสะท้อนตัวเอง)



- **ผู้ขับเคลื่อน:** วงจรการเรียนรู้ภายในของโมเดลเอง (Internal Loop)
- **จุดเด่น:** ขยายสเกลได้ทันที, ออกแบบมาเพื่อจุดอ่อนของโมเดลนั้นๆ โดยเฉพาะ (Model-specific)

กลไกขับเคลื่อน: The Self-Harness Loop

เปลี่ยน "ความล้มเหลว" ให้เป็น "การแก้ไขโค้ด" ผ่านกระบวนการวนซ้ำ 3 ขั้นตอน โดยใช้โมเดลเดิมและ Evaluator เดิม

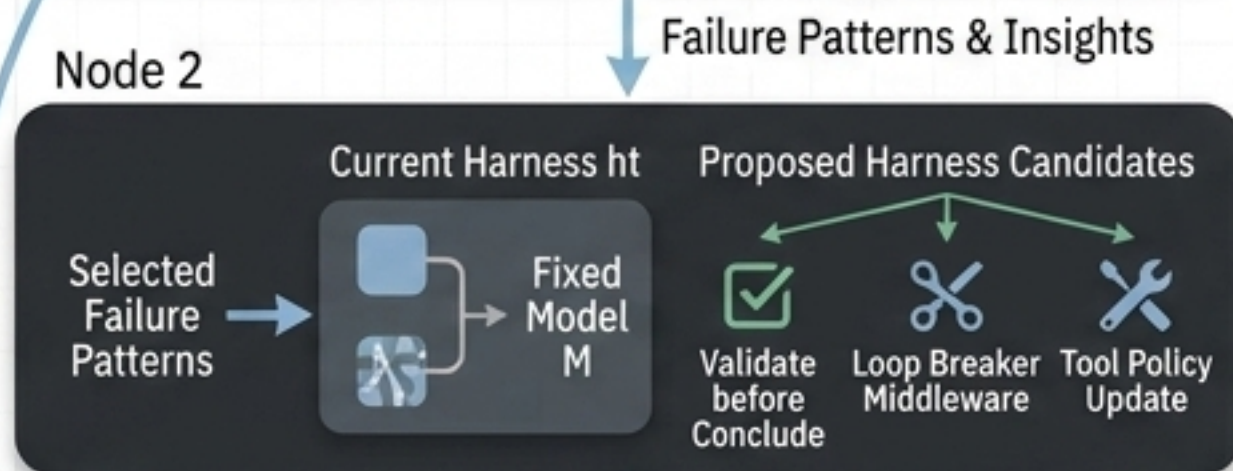
Stage 1: Weakness Mining (การค้นหาลจุดอ่อน)

วิเคราะห์ Execution Traces เพื่อจัดกลุ่มและหา
แพทเทิร์นของข้อผิดพลาด



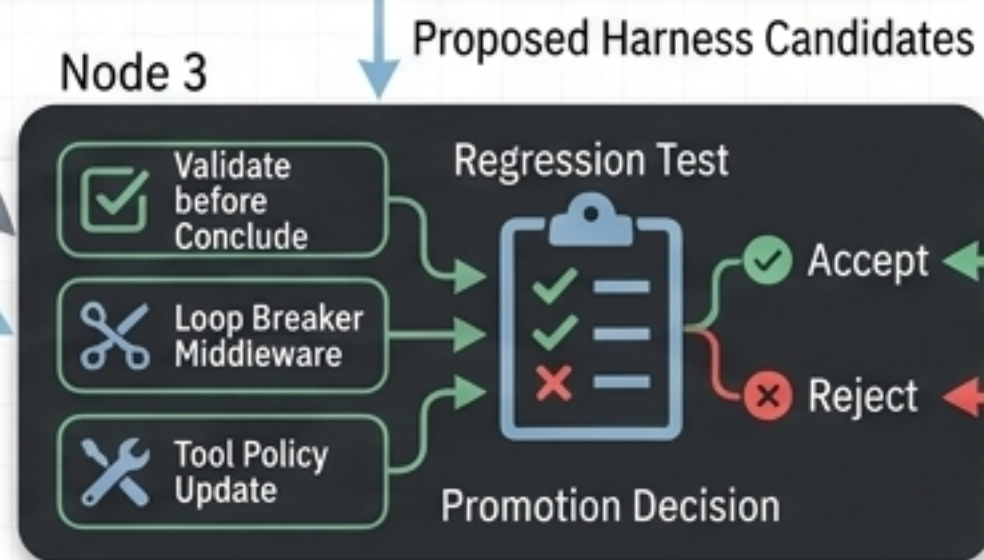
Stage 2: Harness Proposal (การเสนอแนวทางแก้ไข)

เสนอทางออกหลายรูปแบบ (K proposals)
ที่แก้ไขจุดอ่อนนั้นๆ โดยปรับเปลี่ยนโค้ด
ให้น้อยที่สุด (Minimal edits)



Stage 3: Proposal Validation (การทดสอบและยืนยัน)

คัดกรองผ่าน Regression Test อย่างเข้มงวด
ยอมรับเฉพาะการปรับปรุงที่ไม่ทำให้ระบบเดิมพัง



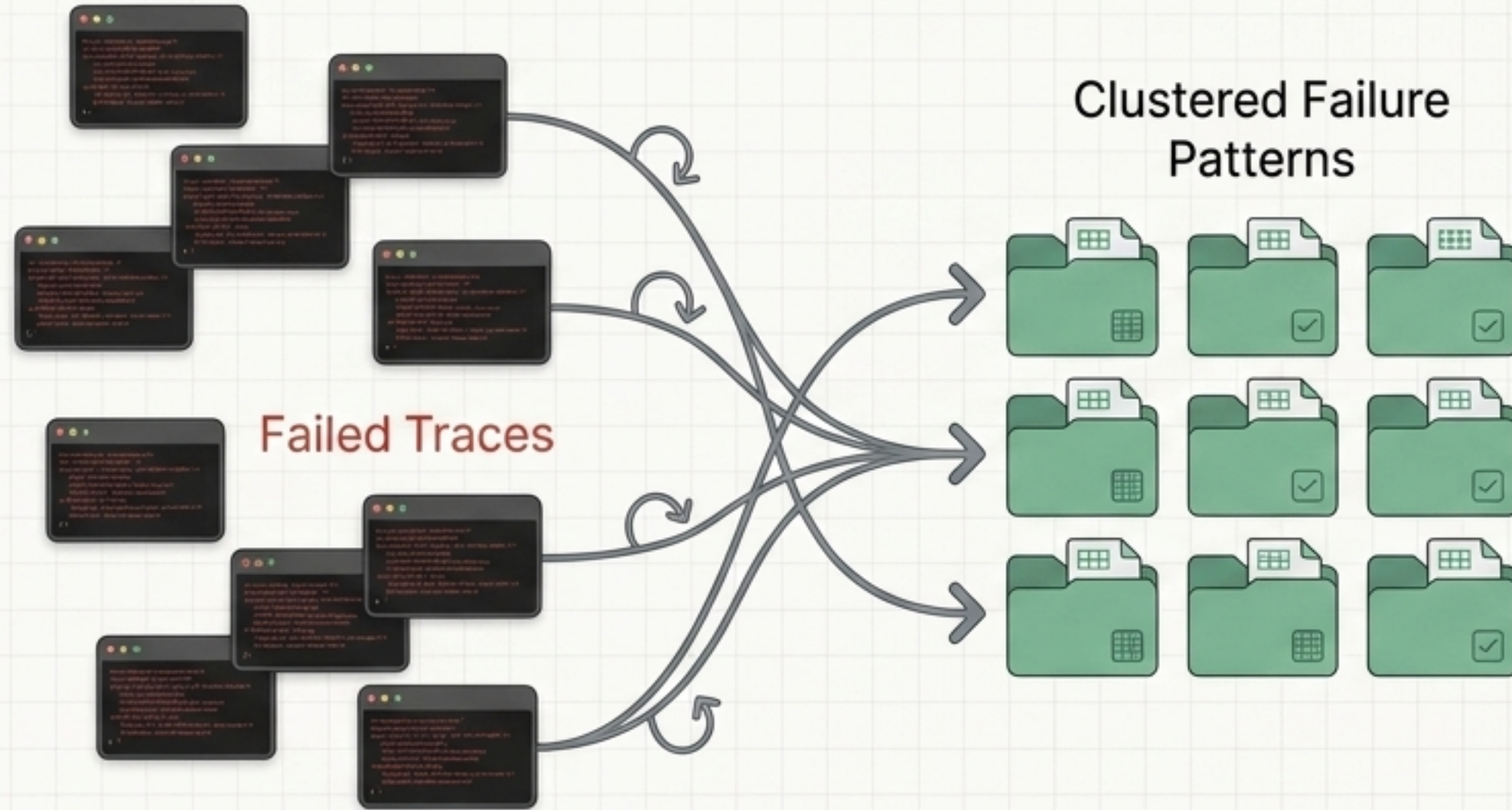
Updated Harness



Proceed to
Next Iteration

Stage 1: Weakness Mining (ค้นหารูปแบบ ไม่ใช่ข้อผิดพลาดประปราย)

โมเดลรันคำสั่งภายใต้ Harness เริ่มต้น และล้มเหลว
ระบบจะไม่มองความล้มเหลวเป็นเหตุการณ์เดียว แต่จะนำ Execution Traces มาจัดกลุ่ม (Cluster)



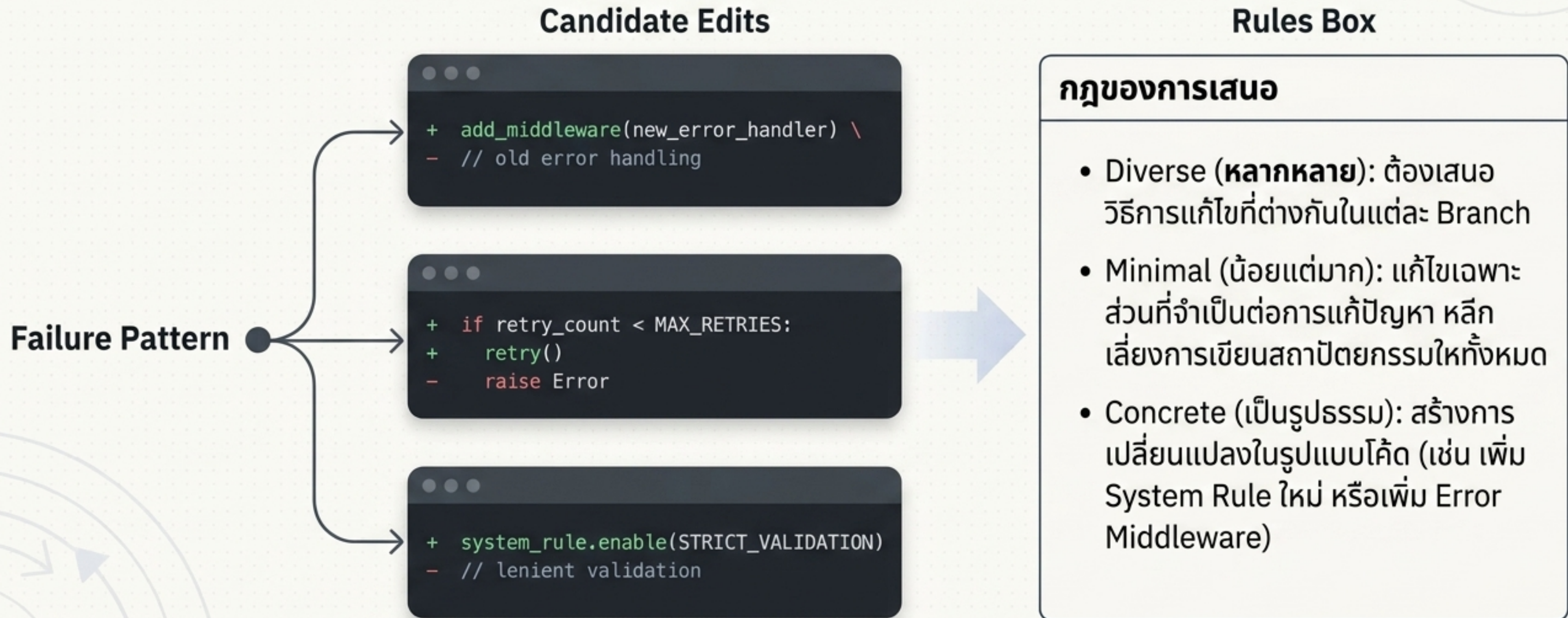
จัดกลุ่มผ่าน Verifier: จัดกลุ่มตามสาเหตุ
หลักจาก Verifier + พฤติกรรมของ Agent

ผลลัพธ์: ได้ Evidence Bundle หรือชุด
หลักฐานที่ระบุกลไกความผิดพลาดที่เกิดขึ้น
ซ้ำๆ (Reusable failure mechanisms)
เพื่อส่งต่อให้ Stage 2

หัวใจสำคัญ: แยกอาการปลายเหตุ (เช่น
Timeout) ออกจากสาเหตุที่แท้จริง (เช่น
วนลูปอ่านไฟล์เดิมซ้ำๆ)

Stage 2: Harness Proposal (ความคิดสร้างสรรค์ภายใต้กรอบ)

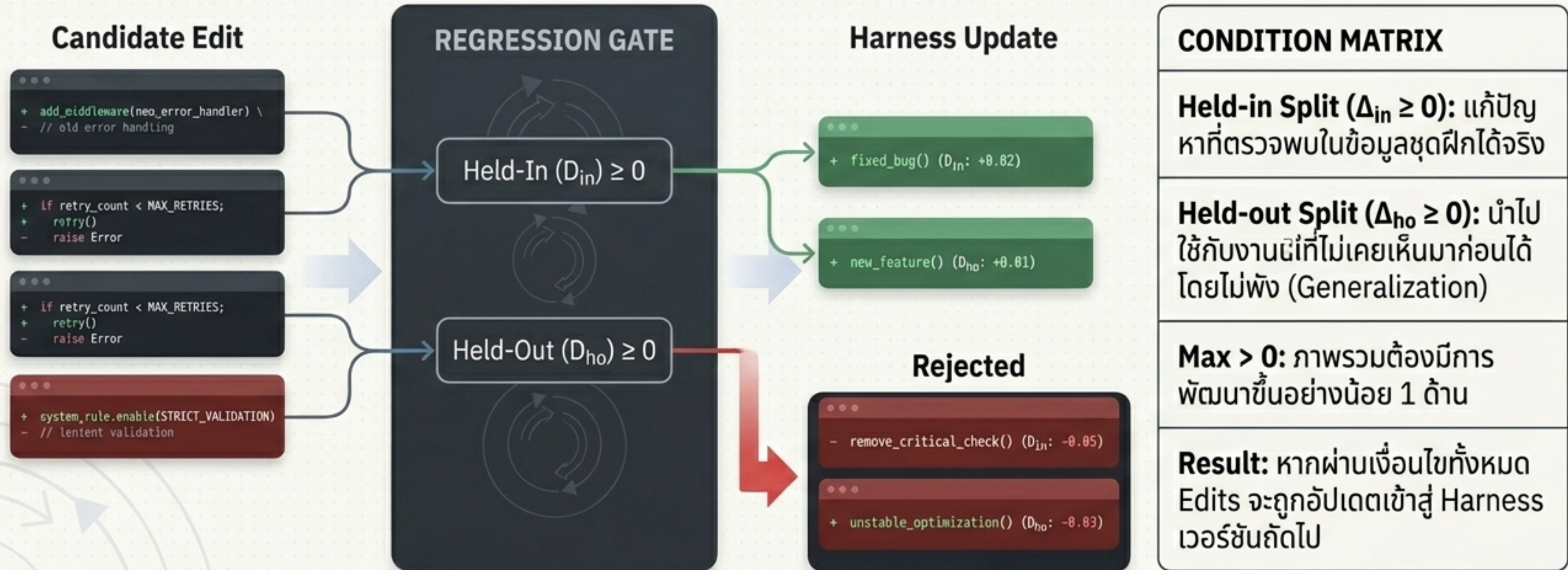
โมเดลตัวเดิมสวมบทบาทเป็น **"Proposer"** (ผู้เสนอ) รับ Evidence Bundle และสร้างทางเลือกในการแก้ไข K รูปแบบ



Stage 3: Proposal Validation (ด่านทดสอบที่ไร้ความปราณี)

ทางออกที่ถูกเสนอจะไม่ถูกนำไปใช้ทันที แต่ต้องผ่าน **Regression Testing** อย่างเข้มงวด เพื่อให้แน่ใจว่าการอัปเดตไม่ทำให้ประสิทธิภาพเดิมลดลง

Logic Gate



สนามทดสอบ: Terminal-Bench-2.0

Environment:

คอนเทนเนอร์ระบบปฏิบัติการจริง ทดสอบการรับคำสั่ง Terminal, จัดการไฟล์, และแก้ไข Error แบบ deterministic

Minimal Baseline:

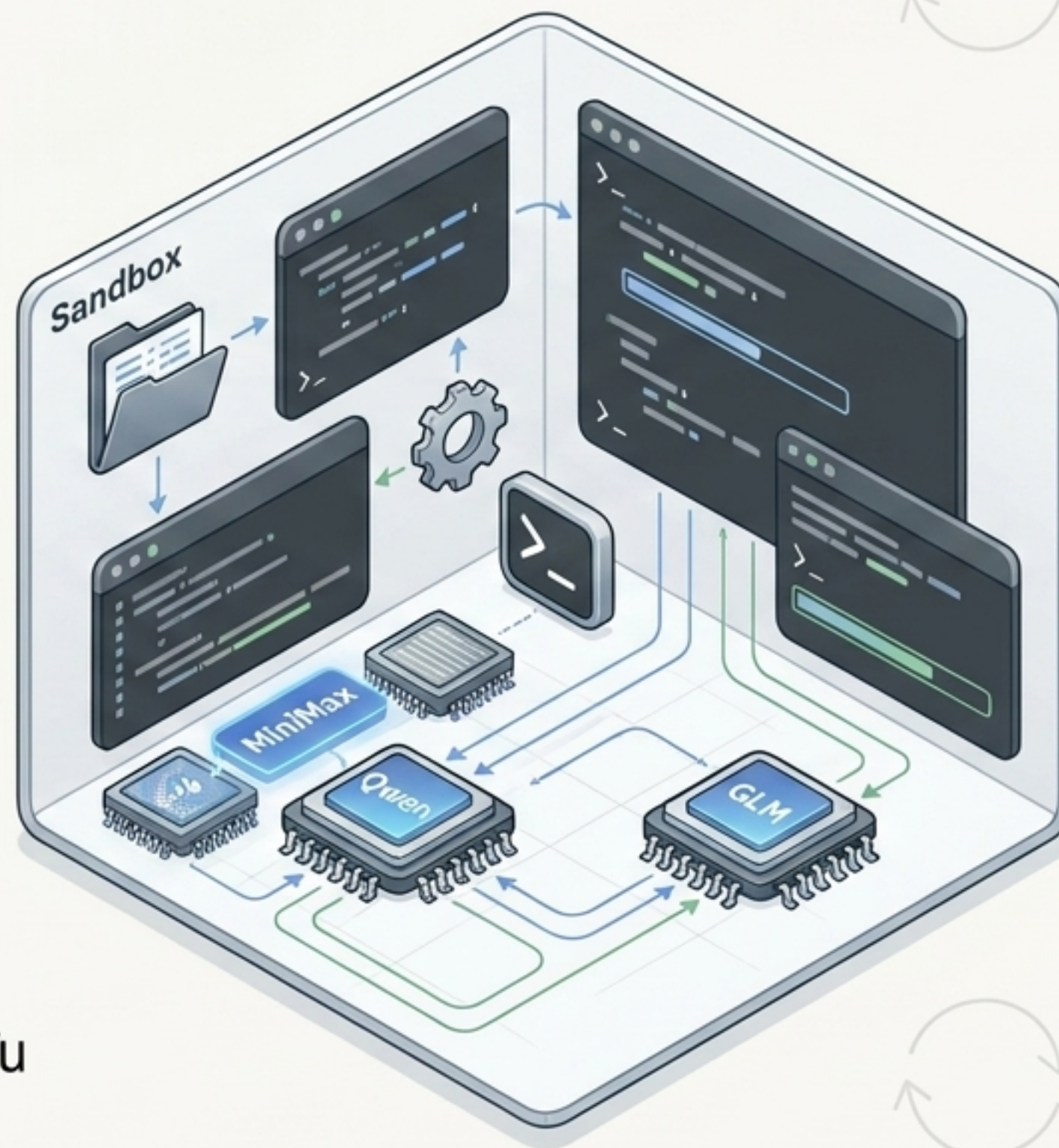
เริ่มต้นจาก Harness ที่เล็กที่สุด ([DeepAgent-based](#)) มีเพียง System Prompt พื้นฐานและเครื่องมือจัดการไฟล์/เชลล์ธรรมดา

3 Diverse Models:

1. MiniMax M2.5
2. Qwen3.5-35B-A3B
3. GLM-5

Objective:

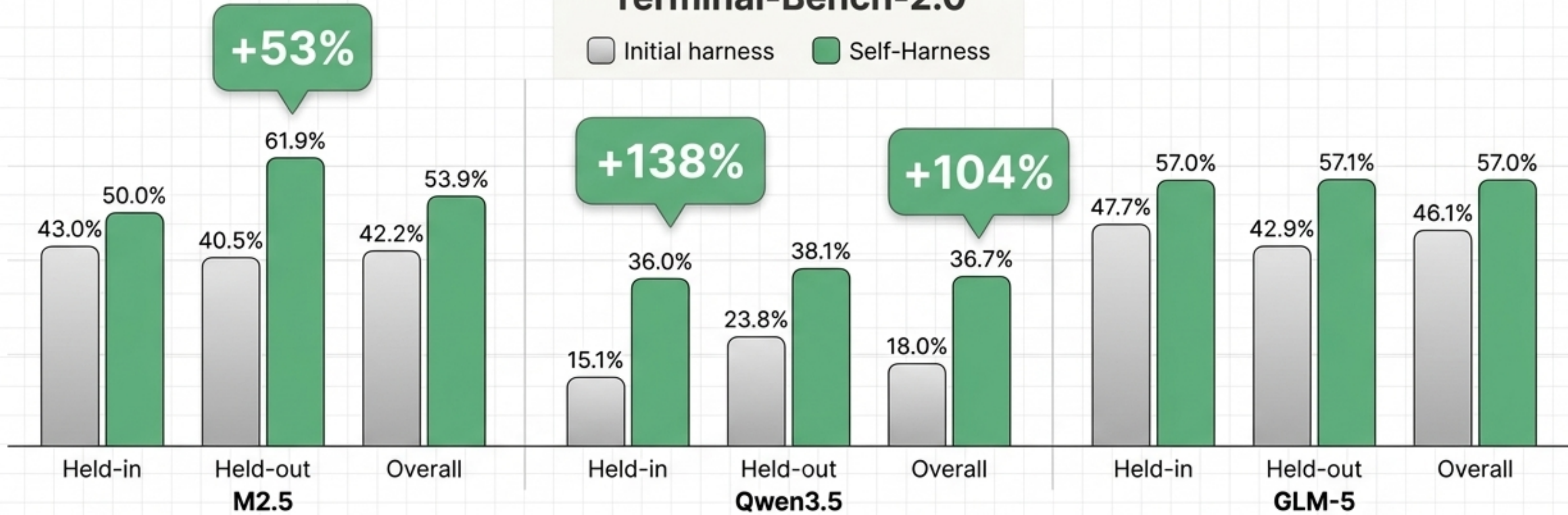
พิสูจน์ว่า [Self-Harness](#) สามารถพัฒนาโมเดลที่มีสถาปัตยกรรมต่างกัน ให้ทำงานได้ดีขึ้นด้วยการแก้ไขจากภายใน



ผลลัพธ์เชิงปริมาณ: ก้าวข้ามขีดจำกัดของ Training Set

Terminal-Bench-2.0

Initial harness Self-Harness



Key Insights

การเติบโตแบบก้าวกระโดด: Qwen3.5 มีประสิทธิภาพใน **Held-in** เพิ่มขึ้นถึง **+138%** (จาก 15.1% เป็น 36.0%)

พิสูจน์การเรียนรู้ (Held-Out Gains): MiniMax M2.5 ผ่าน Held-out ทะยานจาก **40.5%** สู่ **61.9%** (+53%)

ข้อสรุปที่ได้: Agent ไม่ได้แค่ “ท่องจำ” คำตอบ เพื่อผ่านการประเมิน แต่ Self-Harness ทำให้พวกมันเรียนรู้ **“Workflow การทำงานที่ดีขึ้น”** (Generalizing beyond the training set)

บทสรุปเชิงโครงสร้าง: ทำไม "เลื้อยโหล" ถึงใช้ไม่ได้กับ AI Agent

Model	Discovered Weakness	Self-Proposed Fix
MiniMax M2.5	จุดอ่อนที่พบ: ติดลูปการใช้ Tool ซ้ำๆ จนหมดเวลา	Self-Harness Edit: กำหนด Policy: "หากเรียก Tool ครบ 50 ครั้ง ให้หยุดพักสรุปข้อมูลและเปลี่ยนวิธีการ"
Qwen3.5	จุดอ่อนที่พบ: พยายามรันคำสั่งที่ ล้มเหลวแบบตาบอด	Self-Harness Edit: สร้าง Error Middleware: "ถ้าไฟล์ไม่มีอยู่จริง ห้ามค้นหาต่อ ให้สร้างไฟล์นั้นทันที"
GLM-5	จุดอ่อนที่พบ: เสียเวลาสำรวจ Environment มากเกินไป	Self-Harness Edit: เพิ่มคำสั่ง: "เปลี่ยนผ่านจากการ Explore ไปสู่ กระบวนการ Implement และ Test อย่างรวดเร็ว"

หลักฐานระดับ Micro: MiniMax M2.5

การบังคับสร้าง Artifact (Output File) ตั้งแต่เนิ่นๆ

โมเดลเสียเวลาอ่าน Metadata ซ้ำๆ
จนเวลาหมด (Agent timed out after 900s)
ไฟล์ ``/app/answer.txt`` ไม่เคยถูก
สร้าง

ภายใต้กฎใหม่ โมเดลวิเคราะห์ข้อมูล
คำนวณเสร็จ และเขียนไฟล์
``/app/answer.txt`` ทันที
จากนั้นจึงอ่านกลับเพื่อทำการ
Verification

Cognitive Shift: เปลี่ยนจากการ "สำรวจข้อมูลแบบไร้จุดหมาย" เป็น "เป้าหมายอยู่ที่ผลลัพธ์ (Deliverable-focused)"

หลักฐานระดับ Micro: Qwen3.5

การกู้คืนจาก Error ผ่าน Middleware (Error Recovery)

เมื่อพบ Error ขณะแก้ไขไฟล์ `extract.js`
โมเดลหยุดหจิดและลบไฟล์ทิ้ง (`rm /app/extract.js`) ทำให้ Verifier
ตรวจไม่พบไฟล์ตอนจบ

เมื่อ Middleware ที่โมเดลสร้างขึ้นเองตรวจพบ
Tool Error มันจะแทรก Prompt จุกเงิน:
"ตรวจพบไฟล์หาย ให้สร้างใหม่ทันที"

โมเดลจึงกู้คืนไฟล์ (`cat > /app/extract.js`)
และทำงานต่อจนจบ

Cognitive Shift: เปลี่ยนจากการ "ลุยทำต่อแบบตาบอด"
เป็นการ "ตระหนักรู้และแก้ไข Error ทันที (Self-Correction)"

หลักฐานระดับ Micro: GLM-5

การรักษาสภาพแวดล้อม (Environment Persistence)

โมเดลพลาญ Budget ไปกับการพยายามดาวน์โหลดไฟล์ขนาดใหญ่ที่ไม่มีอยู่จริง และเพิกเฉยต่อ Sanity Checks ที่แจ้งเตือน Error

กฎใหม่บังคับให้แบ่งการทำงานเป็นสแต็ป ตรวจสอบ Path และรักษาภาพตัวแปร Environment (Persistence) โมเดลทำการเช็ค URL ก่อนดาวน์โหลด และแก้ไข Error ก่อนสรุปผล

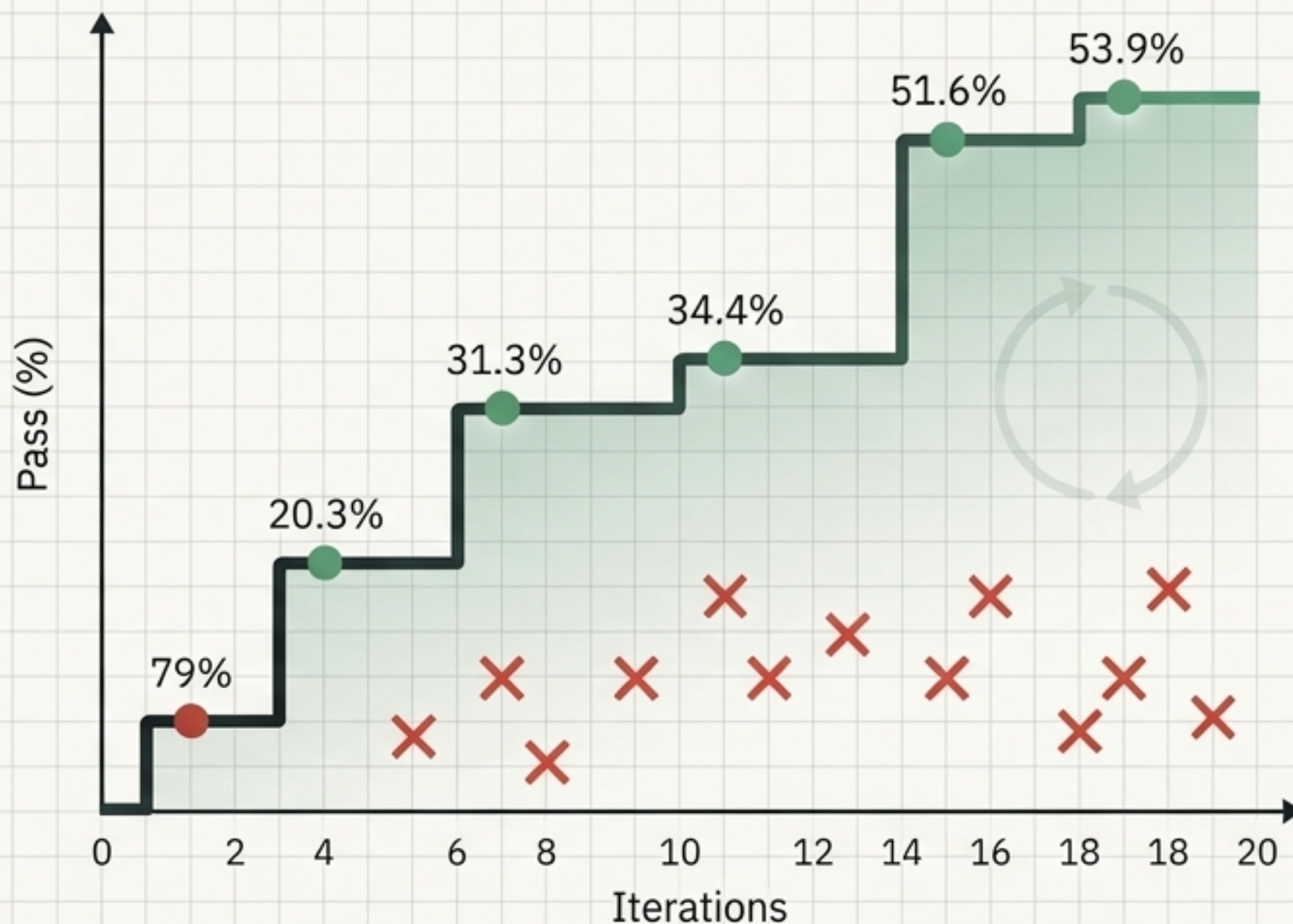
Cognitive Shift: เปลี่ยนจากการ "สุมดาวน์โหลดและหวังผล" เป็นการ "รับคำสั่งเชิงโครงสร้างและตรวจสอบทีละขั้น"

วิวัฒนาการแบบบันได (The Evolutionary Trajectory)

Self-Harness ไม่ใช่ยาวิเศษที่ทำให้เก่งในข้ามคืน แต่เป็น กระบวนการลองผิดลองถูกที่ขับเคลื่อนด้วยหลักฐาน

● **Green Dots (Accepted):** แนวคิดที่พิสูจน์แล้วว่าดีขึ้นทั้ง In-sample และ Out-of-sample (เช่น Middleware ก้นเหินยว, กฎการจำกัดลูป)

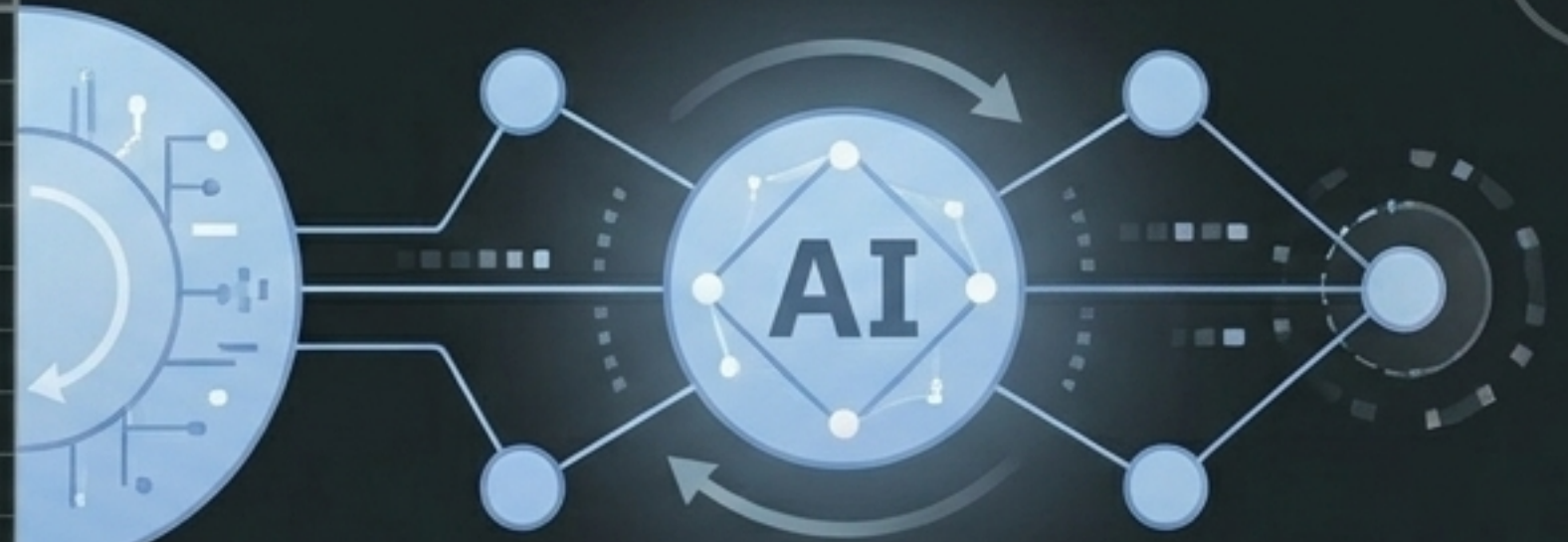
✗ **Red Xs (Rejected):** แนวคิดแย่ๆ (Bad Ideas) จะถูกปิดตกโดยอัตโนมัติด้วย Regression Gate อย่างไร้ความปราณี



การวิวัฒนาการต้องอาศัยกลไกคัดกรองที่เข้มงวด การลองผิดลองถูกอย่างมีระบบคือเบื้องหลังของ AI ที่พัฒนาตัวเองได้

อนาคตของ AI: Agents ในฐานะ Co-Designers

```
class AgentHarness {  
  def adapt_structure(self, trace: Trace) -> Structure:  
    agent.action()  
    agent.action = Agentaction()  
    agent.outcome = Structure_false()  
    return ...  
}
```



โมเดลที่แตกต่าง
ต้องการ Harness ที่แตกต่าง:
หมวดหมู่ของการเขียนโครงสร้างครอบคลุม
จักรวาลแบบ "One Size Fits All"

เรียนรู้จากพฤติกรรม
ไม่ใช่แค่ Prompt:
การพัฒนาระบบที่ยั่งยืนต้องดึงข้อมูล
จาก Execution Traces
มาวิเคราะห์หาจุดอ่อนที่แท้จริง

อิสรภาพภายใต้กรอบการทดสอบ:
Agent สามารถแก้ไขตัวเองได้
แต่ต้องถูกกำกับด้วย Validation Gate
และ Regression Test เสมอ

AI Agent ในอนาคตจะไม่ใช่เพียงระบบที่ถูกมนุษย์สร้างครอบควบคุม (Shaped by humans)
แต่จะมีอิสระและความสามารถในการรู้สร้างและปรับแต่งโครงสร้างการทำงานของตนเอง (Reshape themselves)